metacharacters in regular expressions), and of course %[0-9] matches "any decimal digits." Notice that all elements in the returned array have the string data type: by using regular expressions, sscanf() no longer has information about the required data types of the matched portions of the input.

I could go on and on about sscanf(). Its conversion specification syntax, combined with the ability to use regular expressions, makes it a complex and powerful beast. But the fact that it accepts regular expressions begs the question – why not just use a regular expression function? sscanf() offers no extra benefit over its counterpart preg_match() (which will happily extract data for you and assign the results elsewhere) other than that it is possible to have fine control over data types, and in any case, type casting after extraction with preg_match() is very straightforward. sscanf() has also historically been the source of buffer overflow exploits in PHP (and other languages based on the standard C implementation).

My advice is to stick to preg_match() (which is a far more conventional tool for this job), but be aware that you might encounter sscanf() on your travels through legacy codebases. preg_match() and its brethren are covered in the upcoming section on regular expressions. For reference, you can find the "full" documentation for sscanf() at https://www.php.net/manual/en/function.sscanf.php, and if you insist on using this function with regular expression specifiers, you'll need to trawl the usual places (Stack Overflow, etc.) for help and may God have mercy on you.

Extracting Sub-strings

Table 8-13. *Sub-string extraction functions*

strstr(), mb_strstr() stristr(), mb_stristr() strrchr(), mb_strrchr() mb_strrichr()	Extract up to (or from) first or last occurrence of sub-string
strpbrk()	Return portion of string beginning with any char of a set
strtok()	Extracts portion of string up to first occurrence of a token character
substr(), mb_substr()	Return specific amount of bytes starting at an offset
mb_str_cut()	Byte-level variant of mb_substr(), (but still multi-byte safe)